

Article

Context-Aware Code Review Automation: A Retrieval-Augmented Approach

Büşra İçöz ^{1,2,*}  and Göksel Biricik ¹ 

¹ Department of Computer Engineering, Yildiz Technical University, 34220 Istanbul, Turkey; gbiricik@yildiz.edu.tr

² ING Netherlands, 1102 CT Amsterdam, The Netherlands

* Correspondence: busra.icoz@std.yildiz.edu.tr

Abstract

Manual code review is essential for software quality, but often slows down development cycles due to the high time demands on developers. In this study, we propose an automated solution for Python (version 3.13) projects that generates code review comments by combining Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG). To achieve this, we first curated a dataset from GitHub pull requests (PRs) using the GitHub REST Application Programming Interface (API) (version 2022-11-28) and classified comments into semantic categories using a semi-supervised Support Vector Machine (SVM) model. During the review process, our system uses a vector database to retrieve the top- k most relevant historical comments, providing context for a diverse spectrum of open-weights LLMs, including DeepSeek-Coder-33B, Qwen2.5-Coder-32B, Codestral-22B, CodeLlama-13B, Mistral-Instruct-7B, and Phi-3-Mini. We evaluated the system using a multi-step validation that combined standard metrics (BLEU-4, ROUGE-L, cosine similarity) with an LLM-as-a-Judge approach, and verified the results through targeted human review to ensure consistency with expert standards. The findings show that retrieval augmentation improves feedback relevance for larger models, with DeepSeek-Coder's alignment score increasing by 17.9% at a retrieval depth of $k = 3$. In contrast, smaller models such as Phi-3-Mini suffered from context collapse, where too much context reduced accuracy. To manage this trade-off, we built a hybrid expert system that routes each task to the most suitable model. Our results indicate that the proposed approach improved performance by 13.2% compared to the zero-shot baseline ($k = 0$). In addition, our proposed system reduces hallucinations and generates comments that closely align with the standards expected from the experts.

Keywords: code review; code review automation; Large Language Models (LLMs); Retrieval-Augmented Generation (RAG); software automation; semi-supervised learning



Academic Editors: Linda Vickovic
and Maja Braovic

Received: 26 December 2025

Revised: 22 January 2026

Accepted: 9 February 2026

Published: 13 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Modern software engineering relies heavily on code review, not only to catch defects early, but also to enforce coding standards and prevent security issues from reaching production [1]. Empirical studies on Modern Code Review (MCR) in major technology companies, such as Google, identify rigorous peer review as the main driver for maintaining long-term software quality [2].

Despite its critical importance, the manual nature of this process creates a significant bottleneck. As codebases expand, the cognitive load on developers also increases. Research

indicates that engineers dedicate approximately six hours per week exclusively to code review, wasting valuable time on feature development [3]. Human reviews are subjective; the quality of feedback depends on the expertise of the engineer making the review, and this can often lead to minor logical errors that are overlooked in inconsistent assessments [4].

Although the industry has adopted static analysis tools (SAST) such as SonarQube or ESLint to mitigate these issues, their impact is limited. They are effective at catching syntax violations, but these rule-based systems lack a semantic understanding of business logic and cannot offer context-sensitive suggestions [5]. The recent emergence of Large Language Models (LLMs) offers a promising alternative that demonstrates advanced capabilities in code summarization and generation [6]. However, deploying standard LLMs for code review remains problematic. Without access to a project's specific history or conventions, these models frequently generate generic feedback or hallucinations—reasonable but incorrect recommendations that fail to align with project requirements [7].

In this study, we introduce a Python code review automation framework implemented via Retrieval Augmented Generation (RAG). Unlike generic retrieval approaches that treat all code changes uniformly, our system employs a two-stage process. First, it classifies the pull request into a semantic category (e.g., Functional, Refactoring) to determine the reviewer's intent. Crucially, we implement a dynamic model routing strategy: based on this classification, the system directs the prompt and retrieved context to the specific open-weight LLM architecture (e.g., Qwen2.5-Coder:32B for functional defects, Mistral-Instruct:7B for documentation) that is empirically optimized for that type of category. This structured context and specialized routing enable the selected expert LLM to generate precise, consistent, and actionable feedback. Specifically, this study addresses the following Research Questions (RQs):

- RQ1: To what extent does Retrieval-Augmented Generation (RAG) improve the specificity and accuracy of automated code reviews compared to baseline Large Language Models?
- RQ2: What is the optimal retrieval depth (k) to balance context availability with cognitive load, and at what point does information overload occur?
- RQ3: Do specific LLM architectures demonstrate specialized capabilities across different semantic categories, such as detecting functional bugs versus suggesting refactoring?
- RQ4: To what extent does a dynamic Expert Routing mechanism that assigns tasks to specialized architectures based on semantic category enhance the overall technical accuracy of automated code reviews compared to zero-shot baselines?
- RQ5: How do traditional n -gram metrics (e.g., BLEU, ROUGE) compare to semantic verification methods (such as LLM-as-a-Judge) in accurately assessing the quality of automated code reviews?

Motivated by these questions, our primary contributions to this study are as follows:

- We constructed a specialized dataset of Python code reviews and classified them into semantic categories: Functional, Refactoring, Documentation, and Discussion, based on the taxonomy established by Turzo et al. [8], using a semi-supervised SVM approach.
- We developed a RAG-based pipeline that uses top- k retrieval to guide the LLM, significantly suppressing hallucinations compared to zero-shot baselines.
- We benchmarked our system against human-written ground truth using both traditional metrics and a semantic judge, demonstrating that retrieval-augmented models produce comments with significantly higher relevance and specificity.

The remainder of this paper is organized as follows. Section 2 reviews the literature on MCR and LLM-based automation. Section 3 details our data curation process, classification

strategy, and RAG architecture. Section 4 outlines the experimental setup, including the models and metrics used. Section 5 presents a comprehensive analysis of the results. Finally, Section 6 discusses threats to validity, and Section 7 concludes the study with directions for future work.

2. Related Work

In this section, we summarize the evolution of code review research from manual methods to automated systems. We divide our review into three main parts: the evolution of automated code review, the application of Large Language Models (LLMs) in software engineering, and the integration of Retrieval-Augmented Generation (RAG) for these tasks.

2.1. The Evolution of Code Review Automation

We first examine the origins of code review. This process began with Fagan's code inspection, which established peer review as a critical method for identifying defects [9]. While manual review is effective, we note that it is labor-intensive and often causes delays due to the limited availability of developers.

To address these bottlenecks, the industry first adopted Static Analysis Tools (SAST). These tools automatically detect syntax errors. However, as Johnson et al. pointed out, these systems often generate false alarms and lack a semantic understanding of the code's business logic [5].

Following this, researchers developed Context-Aware Recommender Systems (CARS). We observe that these systems focus on optimizing the process rather than the content. Mateos and Bellogín note that including spatial and temporal context improves the precision of recommendations [10]. Similarly, Sadman et al. introduced the ADCR tool to assign reviewers based on expertise, while Strand et al. confirmed that such context-aware assignments reduce review turnaround times in industrial settings [11].

Eventually, research shifted toward fully Automated Code Review (ACR). Tufano et al. treated code review as a translation task, training models to "translate" buggy code into fixed code [12]. Building on this, Li et al. introduced CodeReviewer, a model designed to understand code changes [13]. We also highlight Lin et al., who argued that automated systems perform better when trained on data filtered for reviewer expertise [14].

Although these models were effective, the rise of Large Language Models (LLMs) offered a more powerful and general solution. This research laid the foundation for using LLMs in code review, which we discuss in the next section.

2.2. Large Language Models for Software Engineering

Large Language Models have significantly improved the automation capabilities of software engineering processes. Unlike previous deep learning models, LLMs can interpret complex logic and provide explanations in natural language. Rasheed et al. showed that LLM-based tools can detect code smells and vulnerabilities that traditional tools do not detect [15]. In addition, Rybalchenko and Al-Turany proposed Pearbot, which uses a multi-agent architecture to simulate collaborative reviews and identify issues that single models might overlook [16].

However, we identify a critical limitation: LLMs often lack project-specific context. Li et al. attempted to solve this by using "code slicing" to provide relevant variable definitions [13]. Haroon et al. also emphasized the need to verify whether LLMs truly understand the code or are just hallucinating [17]. These challenges motivate the need for retrieval-based approaches, which we discuss next.

2.3. RAG for Code Review Tasks

Finally, we examine Retrieval-Augmented Generation (RAG). Lewis et al. define RAG as a framework that retrieves external data to ground the responses of the model, thus reducing hallucinations [18].

In the context of code review, Hong and Baik developed RAG-Reviewer. They used retrieval to find similar historical reviews, which helped the model predict rare, domain-specific technical terms [19]. RAG mitigates the knowledge gap by retrieving external data to ground the model's responses. Wang et al. demonstrated in the CODERAG-BENCH study that RAG significantly boosts performance in code generation tasks by providing the necessary context [20].

2.4. Summary and Research Gap

In summary, while prior studies have successfully applied LLMs to code review, we observe that they mostly rely on generic models or static retrieval methods. Current systems, such as RAG-Reviewer [19], treat all code changes uniformly. They lack the flexibility to adapt to the specific intent of a modification, such as distinguishing between a complex bug fix and a documentation update. In this study, we address this limitation by introducing a dynamic routing strategy. Unlike previous approaches, we first categorize the code change and then route it to the most suitable expert model, ensuring that the retrieval process is optimized for the specific task type.

3. Methodology

We propose a framework to automate code review by augmenting LLMs with domain-specific context. The architecture (Figure 1) operates as an event-driven pipeline from a developer's Pull Request (PR) to the generated review comments. We also constructed a dataset derived from high-quality open-source repositories (Figure 2).

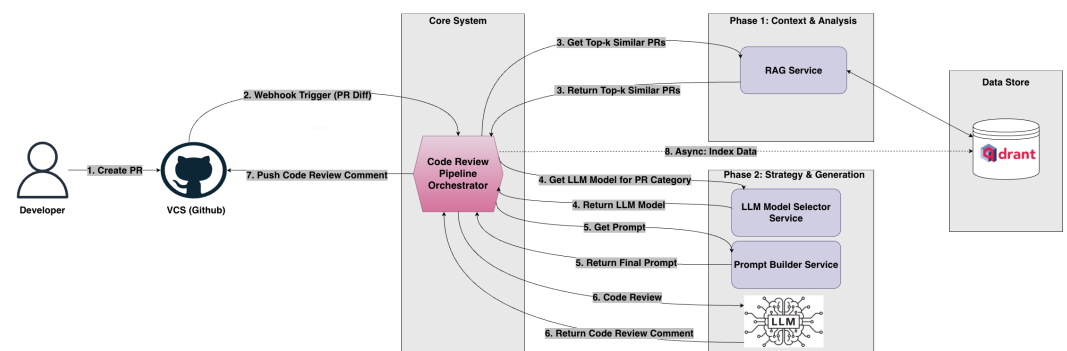


Figure 1. System architecture of the proposed Code Review Pipeline Orchestrator. The orchestrator bridges the developer's VCS (GitHub) and LLMs. Qdrant [21] serves as the vector store for RAG, enabling categorized and strategy-driven code reviews.

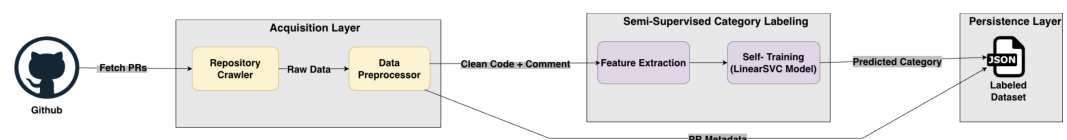


Figure 2. The end-to-end data collection and processing pipeline. The system ingests raw data from GitHub, filters for quality, and applies semi-supervised labeling before indexing.

3.1. System Architecture

Our proposed system works as a middleware layer between the developer's version control system (VCS) and the LLMs, organized into a core workflow plus two phases: Phase 1 (Context & Analysis) and Phase 2 (Strategy & Generation).

3.1.1. The Core Workflow

The process begins when a developer creates a Pull Request (PR), which triggers a webhook event (Steps 1–2). The Code Review Pipeline Orchestrator captures this payload, extracting the code diff, file paths, and metadata. This central component acts as a state manager, routing data between the vector store and the inference engine.

3.1.2. Phase 1: Contextual Retrieval and Semantic Analysis

Standard LLMs often struggle with hallucinations when generating code reviews due to a lack of repository-specific context [22]. To mitigate this, we employ a Retrieval-Augmented Generation (RAG) approach powered by Qdrant [21].

- **Vector Space Modeling:** We embed each code snippet (or diff) using the `sentence-transformers/all-MiniLM-L6-v2` encoder, which generates a 384-dimensional vector representation ($n = 384$) [23]. To measure semantic relatedness between an incoming query vector $\mathbf{A}$ and a stored review vector $\mathbf{B}$, we use cosine similarity, i.e., the cosine of the angle between the two vectors [24,25]:

$$\cos(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}. \quad (1)$$

- **Retrieval-Informed Categorization:** For every incoming PR, the system retrieves the top- k most similar historical examples (Step 3). In this study, we use the term “neighbors” to refer to the vectors in the 384-dimensional embedding space [23] that have the highest cosine similarity to the query vector [24,25]. The system then uses the metadata of these semantic neighbors to infer the category of the new change. For instance, if the majority of the retrieved neighbors are labeled as “Refactoring”, the system uses this as a strong prior for the new Pull Request (PR).
To clearly distinguish the role of categorization in our pipeline: The LinearSVC model (Section 3.4) is employed strictly in the Offline Training Phase to label and expand the historical training dataset. In contrast, during the Online Inference Phase for PR code review, the system utilizes a Retrieval-Informed Categorization strategy. As defined in Equation (1), the semantic category of a new PR is inferred via majority voting from the metadata of the retrieved top- k neighbors ($n = 384$). This ensures that the routing decision is dynamically driven by the most relevant context.

3.1.3. Phase 2: Strategy and Generation

Once the relevant context and inferred category are established, our pipeline moves to the generation phase.

- **Prompt Construction:** The Prompt Builder Service synthesizes a composite prompt (Step 5). This prompt includes the new code diff, the retrieved top- k examples, the inferred category, and subcategory as a strong prior; the complete system prompt template is provided in Appendix A.
- **Guided Generation:** The constructed prompt is sent to the expert LLM (our chosen generator model; details in Section 5). Since the prompt explicitly contains similar historical examples along with their categories, the LLM performs Few-Shot Learning [26]. Using the retrieved examples as a template, it generates a review comment that mimics the style, tone, and technical depth of the retrieved “gold standard” reviews (Step 6). Finally, the generated review comments are pushed back to the VCS (Step 7).

3.1.4. Continuous Feedback Loop (Async Indexing)

To ensure that the system stays in the loop with the project's evolution, we implemented an Asynchronous Indexing Worker (Step 8 in Figure 1). This component runs as a background process to avoid slowing down the main review workflow. Once a PR is successfully merged, this worker captures the final approved code and the reviewer's comments. Then it vectorizes these pairs using the encoder described in Section 3.1.2 and updates the Qdrant index [21]. This decoupling ensures that our knowledge base is continuously refreshed with new "gold standard" (i.e., approved and merged) examples without adding latency to the active pipeline.

3.2. Data Collection and Processing Pipeline

To support the architecture described in the previous section, we developed a robust data pipeline designed for continuous ingestion and automated categorization. The framework consists of three primary layers: Acquisition, Semi-Supervised Labeling, and Persistence, as illustrated in Figure 2.

3.2.1. Dynamic Data Acquisition Strategy

Static datasets in software engineering often become outdated as coding standards and libraries evolve, a challenge highlighted in recent studies on software repository mining [27]. To address this limitation, we designed a Dynamic Data Acquisition Pipeline that autonomously ingests real-world development data via a scheduled crawler. This approach ensures that our retrieval base remains aligned with current Python practices. The dataset is open-source and continuously updated: <https://github.com/busraicoz/crc-py-dataset> (accessed on 26 December 2025).

3.2.2. Continuous Ingestion Heuristics and Temporal Isolation

The crawler employs a Daily Trending Heuristic that queries the GitHub Application Programming Interface (API) every 24 h, targeting the top 10 trending Python repositories in the English language. To ensure the integrity of our evaluation, we apply a strict filtering process:

1. **Temporal Cutoff (Anti-Leakage):** To prevent data contamination, we enforced a strict Temporal Cutoff. We only selected PRs created after 1 June 2024. This date post-dates the knowledge cutoff of the evaluated models, ensuring that the test set consists of unseen data that could not have been memorized during the models' pre-training phase. By targeting only recent contributions that post-date the training cutoffs of the evaluated models, we ensure that our benchmark measures true generalization capability rather than recall.
2. **PR Status:** We only ingest "closed" or "merged" Pull Requests (PRs) to ensure the review cycle is complete, and the code changes are verified.
3. **Comment Selection:** We prioritize the first technical comment in a review thread because subsequent replies often contain conversational text (e.g., "Thanks", "Will fix") rather than independent technical feedback, and filtering these out ensures that the model trains on the primary review instruction.
4. **Data Sanitization:** We filtered the dataset to remove non-informative content. This process excluded automated bot messages and trivial conversational replies (e.g., "Thanks", "Ok") that lack technical substance, adhering to standard data preparation protocols.

3.3. Taxonomy and Semi-Supervised Categorization

We adopted the hierarchical taxonomy established by Turzo and Bosu [8], condensing their 17 subcategories into five high-level intents suitable for retrieval tasks (Table 1). To manage the trade-off between detail and data availability, we employ a hybrid strategy: we use the 17 specific subcategories from the original taxonomy (e.g., “Variable Naming”) to provide detailed context in the system prompt detailed in Section 3.1.3. However, for the routing mechanism detailed in Section 5.4, we aggregate these into 5 high-level categories, which allows for the model to access specific semantic labels without causing data sparsity issues in the routing layer.

Table 1. Code Review Categorization Taxonomy. The high-level categories are optimized to guide the RAG retrieval process.

Category	Description	Mapped Subcategories
Functional	Issues affecting correctness, logic, runtime behavior, or resource management.	Logical errors; Resource handling; Timing; Interface mismatches.
Refactoring	Improvements to code structure, readability, or maintainability.	Variable naming; Code organization; Validation cleanup; Alternate output.
Documentation	Updates to docstrings, inline comments, or external docs.	Documentation.
Discussion	Interactions involving clarification requests or design debates.	Design discussions; Questions; Praise.
False Positive	Invalid concerns or comments explicitly refuted by the author.	False Positive.

3.4. Model Selection and Benchmarking

To automate the labeling of the incoming data category, we benchmarked three different approaches: (1) Zero-Shot classification using GPT-3.5 [26], (2) Logistic Regression with TF-IDF vectors [28], and (3) Linear Support Vector Classification (LinearSVC) with TF-IDF vectors [29].

As presented in Table 2, the LinearSVC approach achieved the highest accuracy (0.622), significantly outperforming the Zero-Shot LLM baseline. Consequently, we selected LinearSVC as the core classifier for our semi-supervised framework.

Table 2. Performance Comparison of Categorization Models. Note: We implemented and evaluated the Zero-Shot LLM [26] and TF-IDF + LogReg [28] baselines specifically for this study to provide a comparative benchmark for our TF-IDF + LinearSVC [29] approach.

Method	Accuracy	F1 Macro	F1 Weighted
Zero-Shot LLM	0.494	0.289	0.479
TF-IDF + LogReg	0.589	0.414	0.609
TF-IDF + LinearSVC	0.622	0.398	0.627

Bold values indicate the best performance in each column.

3.5. Self-Training Algorithm

Using the LinearSVC model, we implemented a self-training loop, as detailed in Algorithm 1, to expand our dataset, a technique proven effective for labeling tasks with limited seed data [30]. The process begins with a manually labeled seed set of 1273 PRs and iteratively labels high-confidence samples from the unlabeled stream.

However, since standard Support Vector Machines (SVMs) output a signed distance rather than a probability, we wrapped the model within a `CalibratedClassifierCV`. This step transforms the raw distance into a calibrated probability distribution (confidence

score). Enabling probability output is essential for our self-training loop [30], as it allows us to strictly apply the confidence threshold ($\tau = 0.7$). Predictions falling below this threshold are discarded to prevent noisy labels from entering the training set.

Algorithm 1 Semi-Supervised Categorization Loop

Require: L : Manually labeled seed set ($N = 1273$)

Require: U : Incoming unlabeled stream from crawler

Require: τ : Confidence threshold (0.7)

Ensure: C : Robust Classifier

```

1: 1. Dual-Branch Feature Extraction
2:  $V_{\text{text}} \leftarrow \text{TF-IDF}(\text{text}, \text{ngram} = (1, 2))$ 
3:  $V_{\text{code}} \leftarrow \text{TF-IDF}(\text{code}, \text{char\_ngram} = (3, 5))$ 
4:  $X \leftarrow \text{Concat}(V_{\text{text}}, V_{\text{code}})$ 
5: 2. Training & Calibration
6:  $C \leftarrow \text{LinearSVC}(\text{class\_weight} = 'balanced')$  ▷ Handle class imbalance
7:  $C_{\text{cal}} \leftarrow \text{CalibratedClassifierCV}(C)$  ▷ Enable probability output
8:  $C_{\text{cal}}.\text{fit}(X_L, Y_L)$ 
9: 3. Pseudo-Labeling & Expansion
10: for  $x_j \in U$  do
11:    $\text{probs} \leftarrow C_{\text{cal}}.\text{predict\_proba}(x_j)$ 
12:   if  $\max(\text{probs}) > \tau$  then
13:      $y_{\text{pred}} \leftarrow \text{argmax}(\text{probs})$ 
14:      $L \leftarrow L \cup \{(x_j, y_{\text{pred}})\}$ 
15:   end if
16: end for
17: return  $C_{\text{cal}}$ 

```

3.6. Data Persistence and Schema Design

To ensure consistent processing across the RAG pipeline, all collected and labeled data are serialized into a standardized JSON format. This schema was designed to preserve critical metadata—such as the repository (repo) origin and the timestamp (created_at)—which provides temporal and project-specific context during the retrieval phase.

As shown in Figure 3, each entry encapsulates the code diff, the reviewer's comment, and the inferred semantic labels. This structured representation facilitates efficient parsing using the embedding models described in Section 3.

<pre> { "code": "@@ -35.11 +35.11 @@\n - except TypeError as te:\n + except TypeError as te: # codespell...", "comment": "you can just use 'e' for these, no need\n to clutter with codespell...", "file_path": "reflex/base.py", "category": "functional", "subcategory": "solution approach", "repo": "reflex-dev/reflex", "pr_number": 4831, "created_at": "2025-02-18T20:50:42Z" } </pre> (a)	<pre> { "code": "@@ -1126.7 +1126.12 @@\n - assert_(sys.getrefcount(ob) == - ... \n + if sys.version_info < (3.14): + \n assert_(newrc == rc - 1)", "comment": "I don't understand why this changed - \n maybe @seberg or @mhvk might have ...", "file_path": "numpy/_core/tests/test_nditer.py", "category": "discussion", "subcategory": "question", "repo": "numpy/numpy", "pr_number": 28748, "created_at": "2025-04-17T15:54:27Z" } </pre> (b)
--	---

Figure 3. Representative JSON entries from the `crc-py-dataset`. The schema standardizes unstructured code reviews into a machine-readable format optimized for vector embedding. (a) Sample A: Functional Category. A variable naming suggestion in the `reflex` repository. (b) Sample B: Discussion Category. An inquiry regarding version-specific behavior in `numpy`.

4. Experimental Design and Evaluation

To validate the efficiency of our proposed framework, we conducted a large-scale comparative study aligned with the defined Research Questions (RQ) in Section 1. Unlike synthetic benchmarks, our experimental setup simulates a real-world enterprise software environment using the Home Assistant project [31] as a test set. This approach allows us to assess model performance within a complex and actively maintained software project, thus replicating the dynamics of real-world collaborative development.

4.1. Dataset Characteristics and Knowledge Base Construction

We focus our evaluation on *Home Assistant Core* (`home-assistant/core`), a modular Python codebase that presents a heterogeneous contribution surface suitable for project-based code review experiments [32].

To simulate a robust Context-Aware AI agent, we built a Composite Knowledge Base designed to capture both project-specific data and general coding best practices. Our retrieval corpus consists of two distinct layers:

1. **Target Project History (Primary Source):** We extracted 3739 historical review pairs directly from the *Home Assistant Core* repository. These samples represent the specific coding style, architectural patterns, and conventions unique to the target project.
2. **crc-py-dataset (Secondary Source):** To address the “cold start” problem—where specific patterns may not yet exist in the project history—we augmented the vector store with our *crc-py-dataset* dataset. This corpus contains high-quality review samples from diverse top-tier open-source repositories to ensure that the model has a fundamental understanding of general software engineering principles.

Retrieval Strategy: We utilize a unified vector space for both data sources. During the inference phase, the retrieval mechanism performs a semantic search across this composite index, establishing an implicit fallback mechanism:

- If the code difference is semantically close to the previous *Home Assistant Core* PR, the system prioritizes retrieving project-specific context, ensuring style alignment.
- If no high-confidence match is found within the project history, the retrieval system naturally falls back to relevant examples from the *crc-py-dataset* dataset.

Data Splitting: The test set remains strictly isolated to prevent data leakage. We enforced a strict chronological split:

- **Knowledge Base (Project Memory):** The training split ($N = 3739$) populates the RAG Vector Store with historically observed review patterns.
- **Test Set (Unseen Contributions):** The test split ($N = 1625$) contains contributions strictly after the cutoff date, emulating deployment on unseen changes [33].

Semantic Distribution Analysis

To understand the complexity of the review task, we analyzed the semantic distribution of the dataset (Figure 4).

As shown in Figure 4, the dataset is oriented mainly toward constructive feedback types (>67% Solution Approach and Question). This distribution justifies our preference for using reasoning-based models such as DeepSeek-Coder-33B and Qwen2.5-Coder-32B. To avoid distorting the assessment, we layered the test set (Table 3) to ensure adequate representation of minority classes such as “False Positive” ($N = 15$).

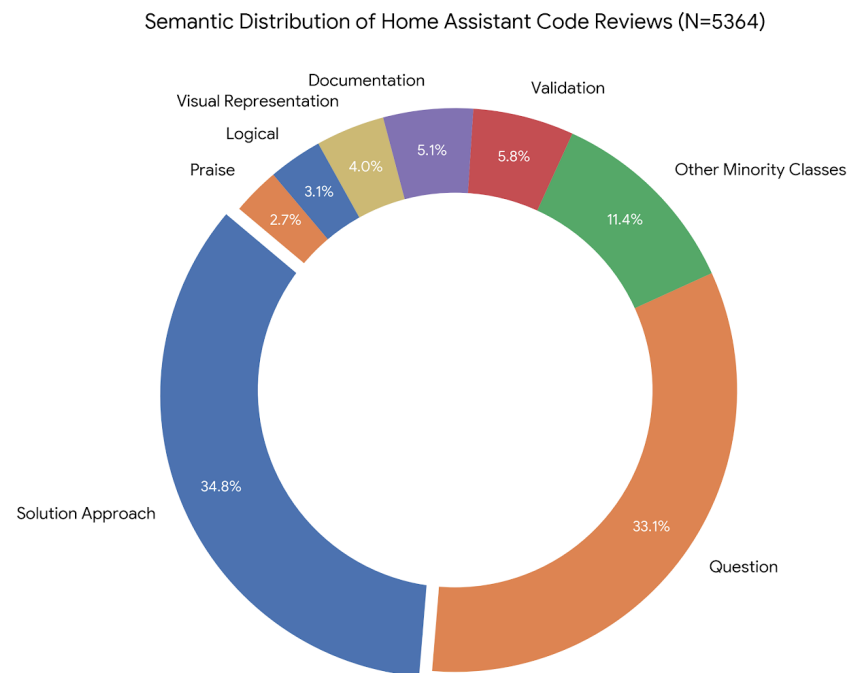


Figure 4. Semantic distribution of the Home Assistant dataset ($N = 5364$). The dominance of “Solution Approach” (34.8%) and “Question” (33.1%) categories shows that models are evaluated on their ability to provide actionable feedback rather than simple validation.

Table 3. Composition of the Evaluation Dataset (test set). We enforced a minimum constraint ($N \geq 15$) for minority classes to ensure rigorous testing of edge cases.

Subcategory	Total Available	Test Set (N)	Test Ratio (%)
Alternate Output	84	40	47.6%
Code Organization	70	25	35.7%
Design Discussion	54	25	46.3%
Documentation	276	75	27.2%
False Positive	17	15	88.2%
Functional	65	25	38.5%
Interface	77	35	42.5%
Logical	166	65	39.2%
Praise	146	65	44.5%
Question	1776	435	24.5%
Resource	20	15	75.0%
Solution Approach	1867	530	28.4%
Support Issues	77	25	45.5%
Timing	105	45	42.9%
Validation	311	115	37.0%
Variable Naming	41	15	36.6%
Visual Representation	212	75	35.4%
Total	5364	1625	30.3%

4.2. Subject Models

To answer RQ1, we selected six state-of-the-art open-weight LLMs (presented in Table 4) based on their rankings on the Hugging Face Open LLM Leaderboard [34] and architectural diversity.

Table 4. Subject Models utilized in the experiment. All models were executed locally via the Ollama framework [35] to ensure reproducibility.

Model	Params	Context	Selection Rationale
DeepSeek-Coder	33B	16k	SOTA Open Code Model [36]
Qwen2.5-Coder	32B	32k	Advanced Reasoning Logic [37]
Codestral	22B	32k	Optimized for Code Completion [38]
CodeLlama	13B	16k	Industry Standard Baseline [39]
Mistral-Instruct	7B	8k	General Purpose Reasoning [40]
Phi-3-Mini	3.8B	4k	Efficiency Benchmark (SLM) [41]

4.3. Experimental Configuration

We manipulate the retrieval strategy as the primary independent variable. We tested four distinct configurations for each subject model:

1. **Baseline** ($k = 0$): The model reviews the code solely based on its pre-trained parametric knowledge (Zero-Shot).
2. **Project-Aware RAG** ($k \in \{1, 3, 5\}$): The model is augmented with the top- k most similar historical reviews, retrieved exclusively from the knowledge base.

To ensure that the observed improvements were due to the RAG mechanism rather than random variance, we fixed the generation temperature to $\tau = 0.1$ and standardized the context window to 4096 tokens across all models.

4.4. Evaluation Methodology

Given the linguistic variability of code reviews, we employed a multifaceted evaluation protocol involving automated metrics, large-scale LLM scoring, and targeted human verification.

4.4.1. Automated Text Similarity Metrics

We use three automatic metrics to compare each generated review comment with its human-written reference. BLEU-4 and ROUGE-L capture surface overlap in different ways, while an embedding-based cosine score provides a coarse semantic signal when multiple phrasings are reasonable [25,42,43]. Because BLEU scores can vary with tokenization and other evaluation settings, we keep the implementation of the metric fixed across all experiments and document the configuration for reproducibility [44].

BLEU-4 is computed as the geometric mean of modified n -gram precisions up to $n = 4$, with a brevity penalty (BP) applied at the corpus level [42]. With uniform weights $w_n = \frac{1}{4}$,

$$\text{BLEU-4} = \text{BP} \cdot \exp \left(\sum_{n=1}^4 w_n \log p_n \right). \quad (2)$$

We use the original definition of the brevity penalty with candidate length c and effective reference length r [42]:

$$\text{BP} = \begin{cases} 1, & c > r, \\ \exp(1 - \frac{r}{c}), & c \leq r. \end{cases} \quad (3)$$

To make BLEU results comparable between runs, we standardized our configuration following the established guidance on BLEU reporting [44].

ROUGE-L is based on the Longest Common Subsequence (LCS), which rewards in-order overlap without requiring contiguous matches [43]. For reference X (length m) and

hypothesis Y (length n), let $\text{LCS}(X, Y)$ denote the LCS *length*. We compute LCS-based recall and precision [43]:

$$R_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{m}, \quad P_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{n}. \quad (4)$$

We report sentence-level ROUGE-L F_1 with $\beta = 1$ (precision and recall weighted equally), following Lin (2004) [43]:

$$F_{\text{LCS}} = \frac{(1 + \beta^2) P_{\text{LCS}} R_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}. \quad (5)$$

We also compute cosine similarity between sentence embeddings as a simple semantic proximity measure [24,25]:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \quad (6)$$

We obtain embeddings using `sentence-transformers/all-MiniLM-L6-v2` (384 dimensions) [23]. We use this score as a supplement to lexical metrics; it is not intended to verify technical correctness on its own.

4.4.2. Large-Scale Semantic Evaluation (LLM-as-a-Judge)

To assess technical precision in our extensive test set ($N = 1625$), we implemented a scalable “LLM-as-a-Judge” protocol [45]. We deployed Llama-3-70B-Instruct [46] as the evaluator to efficiently handle the $>39,000$ inference steps. The judge evaluates each review on a 10-point scale (Table 5).

Table 5. Scoring rubric for the LLM-as-a-Judge protocol, adapted from the MT-Bench methodology [45].

Score	Criteria Definition
10	Perfect: Identifies exact issue and suggests correct fix.
8–9	High Quality: Technically correct, minor tone issues.
5–7	Partial: Identifies topic but solution is incomplete.
1–4	Failure: Misses core issue or hallucinates.
0	Invalid: Factually incorrect or empty.

4.4.3. Human Evaluation and Calibration

To mitigate the inherent bias of automated judge models, we conducted a targeted human verification study. A team of three senior backend engineers, each with over nine years of experience, conducted a manual evaluation of randomly selected samples from the dataset.

Sampling Strategy: We selected at least 5 distinct examples for each of the 17 subcategories listed in Table 3, resulting in a high-quality validation set of 105 samples.

Calibration Protocol: The human reviewer scored the sampled instances using the rubric in Table 5, while remaining blind to the LLM-as-a-Judge outputs. We then quantified the agreement between the human and judge scores. This calibration step was used to assess whether automated ratings reflect human perceptions of review quality, with particular attention to more subjective categories such as “Design Discussion” and “Code Organization”.

5. Experimental Results and Discussion

In this section, we critically evaluated the experimental findings to address the five Research Questions (RQs) proposed in Section 1. We examine the interaction among

retrieval strategies, model architectures, and evaluation metrics, and interpret the findings through the concepts of Information Overload and Semantic Alignment.

5.1. RQ1: Baseline Capabilities and the “Reasoning Gap”

RQ1: To what extent does Retrieval-Augmented Generation (RAG) improve the specificity and accuracy of automated code reviews compared to baseline Large Language Models?

To better understand how each architecture performs without the influence of retrieval, we start by looking at the zero-shot performance ($k = 0$). This baseline shows the knowledge that the model has built up during pre-training. As seen in Table 6, the results highlight a clear ranking in performance, which is driven more by how well the model specializes in the domain rather than just the size of the parameters.

Table 6. Performance comparison across varying retrieval depths (k). We report BLEU-4 and ROUGE-L for lexical overlap, alongside Semantic Similarity, Judge Score (0–10), and Human Evaluation (0–10). Note: All results presented in this table are derived from our experimental evaluation on the Home Assistant Core test set [31,32] as defined in Section 4.1. The models were executed locally using the Ollama framework [35].

Model	k	BLEU-4	ROUGE-L	Sem. Sim	Judge Score	Human Eval.
CodeLlama-13B	0	0.009	0.111	0.318	3.54	3.80
	1	0.007	0.099	0.233	3.60	4.20
	3	0.008	0.104	0.263	3.87	4.80
	5	0.010	0.100	0.245	3.77	4.20
Codestral-22B	0	0.011	0.133	0.397	5.06	6.00
	1	0.014	0.134	0.398	5.47	6.40
	3	0.018	0.141	0.418	5.67	7.20
	5	0.021	0.144	0.416	5.42	6.20
DeepSeek-Coder-33B	0	0.011	0.127	0.329	4.45	5.20
	1	0.011	0.117	0.356	4.67	5.60
	3	0.013	0.123	0.361	5.24	6.40
	5	0.012	0.121	0.358	5.02	6.00
Mistral-Instruct-7B	0	0.012	0.126	0.416	6.01	6.00
	1	0.016	0.129	0.386	5.62	5.80
	3	0.015	0.131	0.388	5.63	5.80
	5	0.017	0.133	0.386	5.46	5.20
Phi-3-Mini (3.8B)	0	0.009	0.123	0.384	5.52	6.00
	1	0.009	0.121	0.359	5.28	5.60
	3	0.010	0.116	0.336	5.23	5.80
	5	0.013	0.112	0.311	4.74	5.00
Qwen2.5-Coder-32B	0	0.020	0.153	0.447	6.34	7.00
	1	0.031	0.161	0.472	6.68	7.40
	3	0.028	0.165	0.478	6.76	7.60
	5	0.028	0.164	0.467	6.54	7.40

Bold indicates the highest score within each model’s group.

Analysis: The strong performance of Qwen2.5-Coder-32B (6.34) demonstrates the effectiveness of zero-shot learning. Unlike general-purpose models, Qwen2.5-Coder-32B has a natural “code intuition” that allows it to identify functional problems without needing external context [39].

Interestingly, there is a gap between lexical fluency and functional accuracy in general models such as Mistral-Instruct-7B. Although Mistral-Instruct-7B scores high in Semantic Similarity (0.416), its technical precision drops when using RAG (6.01 $\rightarrow$ 5.46). We can call this Surface-Level Semantic Alignment: the model may sound like it is providing a solid code review, but it loses its technical accuracy when the context becomes too complex. This

suggests that tuning models for general dialog could unintentionally weaken their ability to maintain technical precision in specialized tasks [47].

5.2. RQ2: Context Saturation and the “Attention Bottleneck”

RQ2: What is the optimal retrieval depth (k) to balance context availability with cognitive load, and at what point does information overload occur?

Our sensitivity analysis, visualized in Figure 5, uncovers two divergent behaviors governed by model scale, confirming the hypothesis that RAG is not a “free lunch” for all architectures [48].

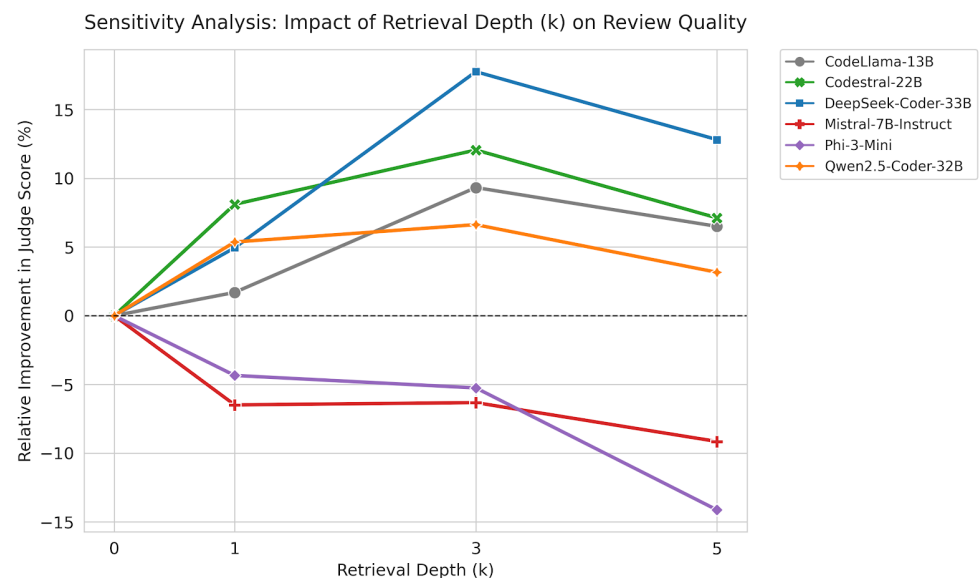


Figure 5. Sensitivity Analysis: Impact of Retrieval Depth (k) on Technical Accuracy. Note the sharp divergence: specialized models (DeepSeek-Coder-33B, Qwen2.5-Coder-32B) benefit from context, whereas smaller models (Phi-3-Mini, Mistral-Instruct-7B) suffer from “Context Collapse” at $k = 5$.

5.2.1. Latent Knowledge Activation

Specialized code models such as DeepSeek-Coder-33B demonstrate a strong positive correlation with context retrieval. DeepSeek’s Judge Score surged by 17.7% at $k = 3$ (from 4.45 to 5.24). We interpret this as Latent Knowledge Activation: the model already possesses the engineering knowledge to review code, but requires project-specific examples to align its output style. For these models, $k = 3$ represents the optimal signal-to-noise ratio.

5.2.2. Context Collapse in SLMs

In contrast, Small Language Models (SLMs) exhibited context collapse. Phi-3-Mini(3.8B) showed a linear degradation ($5.52 \rightarrow 4.74$) as k increased to 5. We attribute this to the Attention Bottleneck: limited capacity models struggle to filter relevant signals from dense historical data [49]. At $k = 5$, the retrieved snippets overwhelmed the model’s attention mechanism, leading to Negative Transfer, where irrelevant context distracted the model from the actual code change.

5.3. RQ3: Architectural Proficiency and Error Severity Overestimation

RQ3: Do specific LLM architectures demonstrate specialized capabilities across different semantic categories?

Based on the findings in RQ2, we selected a retrieval depth of $k = 3$ for this evaluation. This setting offers the best trade-off: it provides sufficient historical context to guide larger models; yet, it avoids the context collapse observed in smaller models when they are

exposed to excessive data. The heatmap in Figure 6 captures these results, showing the comparative quality scores for each model across the tested categories.

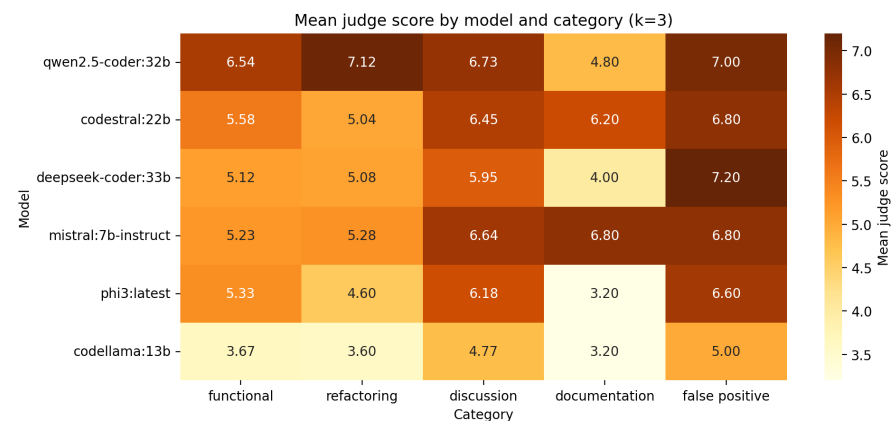


Figure 6. Generation Quality Heatmap (Judge Scores at $k = 3$). Darker cells indicate higher quality. Notice the separation of concerns: “Qwen2.5-Coder-32B” dominates Logic (top-left), while “Mistral-Instruct-7B” dominates Documentation (bottom-right).

Table 7 details the performance breakdown in four semantic categories. The data expose a fundamental trade-off between logical reasoning and language fluency.

Table 7. Model performance (Judge Score 0–10) across semantic categories at $k = 3$. A distinct trade-off is visible: Code-Specialized models (Qwen2.5-Coder-32B) dominate logic tasks, while generic models (Mistral-Instruct-7B) excel in documentation.

Model	Semantic Category ($k = 3$)				Mean
	Functional	Refactoring	Discussion	Docs	
CodeLlama-13B	3.82	3.95	4.10	3.60	3.87
Phi-3-Mini	5.10	5.25	5.40	5.15	5.23
DeepSeek-Coder-33B	5.85	5.90	5.10	4.12	5.24
Mistral-Instruct-7B	5.45	5.50	5.80	6.80	5.89
Codestral-22B	5.90	6.10	6.20	6.45	6.16
Qwen2.5-Coder-32B	6.76	7.20	7.36	4.80	6.53

Bold indicates the best performance in each category.

5.3.1. The Logic-Fluency Dichotomy

“Qwen2.5-Coder-32B” performs very well in reasoning tasks, scoring 7.20 in Refactoring. This suggests that models trained on code execution traces develop a good understanding of Abstract Syntax Trees (ASTs). However, Qwen scores lower in “Documentation” (5.00), while the generalist “Mistral-Instruct-7B” scores better (6.80). This indicates that documentation is mainly a translation task (Code-to-Natural Language) that relies more on language skills than on strict algorithmic reasoning.

5.3.2. Error Severity Overestimation in Reasoning Models

Despite its precision, “Qwen2.5-Coder-32B” exhibits a behavioral misalignment that we define as Error Severity Overestimation. It systematically misclassified 84.6% of benign stylistic suggestions as serious functional bugs. In practice, models optimized for strict code correctness often treat “invalid code” (technical debt) as “broken code” (system failure), which could lead to too many “False Positive” categories in Continuous Integration and Continuous Delivery (CI/CD) pipelines [50].

5.3.3. Quantifying Failure Modes: Hallucination and Severity Overestimation

To further validate our findings, we analyzed the data from our human evaluation ($N = 105$) to quantify two specific failure modes. We defined these metrics using the expert scores and dataset labels as follows:

- **Hallucination Rate:** This assesses the model’s reliability regarding factual accuracy. It is defined as the rate at which responses contain verifiable fabrications (such as non-existent APIs, fabricated code elements, or direct contradictions with the code changes), explicitly excluding generic low-quality advice that is technically correct but unhelpful.
- **Severity Overestimation:** This assesses the model’s tendency to overstate severity. It is defined as the rate at which innocuous changes (such as Refactoring, Documentation, or Visual Representation) are incorrectly classified as “Functional” defects.

Table 8 presents the “Hallucination Rates” and “Severity Overestimation” across different retrieval depths.

Table 8. Failure rates evaluated by human experts ($N = 105$). Metrics include Hallucination Rate and Severity Overestimation, which measure the error rate specifically on the non-functional subset.

Model	Context (k)	Hallucination Rate (%)	Severity Overestimation (%)
DeepSeek-Coder-33B	0	19.19	20.00
	1	20.61	29.52
	3	17.69	19.05
	5	21.43	31.43
Phi-3-Mini	0	10.66	20.95
	1	15.95	20.95
	3	15.00	22.86
	5	20.97	33.33
Codestral-22B	0	16.32	21.90
	1	13.01	20.00
	3	12.52	11.43
	5	14.65	9.52
Qwen2.5-Coder-32B	0	10.23	20.00
	1	9.69	20.00
	3	7.86	11.43
	5	11.47	22.86
Mistral-Instruct-7B	0	9.83	36.19
	1	10.38	37.14
	3	17.30	43.81
	5	15.71	41.90
CodeLlama-13B	0	31.88	36.19
	1	26.29	35.24
	3	29.95	18.10
	5	37.27	41.90

Bold values indicate the lowest (best) error rate within each model’s group.

The results highlight distinct behavioral patterns across the evaluated architectures:

High Precision in Reasoning Models: Qwen2.5-Coder-32B demonstrated the best performance at $k = 3$, with a Hallucination Rate of 7.86% and a Severity Overestimation of 11.43%. This contradicts the assumption that reasoning models are inherently overly strict [50]. It suggests that when provided with sufficient context, these models can accurately distinguish between style suggestions and functional bugs.

Context Collapse in Small Models: Phi-3-Mini showed a clear degradation in performance as we added more context. Its hallucination rate increased from 10.66% at

$k = 0$ to 20.97% at $k = 5$. This aligns with the “Lost in the Middle” phenomenon [49], where smaller models struggle to filter relevant information from noisy retrieval contexts.

Bias in Generalist Models: Mistral-Instruct-7B exhibited a consistently high Severity Overestimation rate ($>36\%$) across all settings. Although this model produces fluent text, it lacks the technical nuance to correctly categorize the severity of defects, often marking minor problems as critical failures.

These quantitative findings support the routing logic of our Hybrid Expert Routing (Section 5.4). Although Qwen2.5-Coder-32B exhibits the highest technical stability (lowest error rates), our earlier analysis (Table 7) revealed that it performs poorly in the “Documentation” category (Judge Score: 5.00). In contrast, Mistral-Instruct-7B achieves a significantly higher quality score (6.80) in that domain. Therefore, our routing strategy prioritizes the model with the highest domain-specific utility: we utilize Qwen2.5-Coder-32B for “Functional” tasks to ensure strict correctness, while routing “Documentation” tasks to Mistral-Instruct-7B to maximize the usefulness and relevance of the feedback.

5.4. RQ4: The Efficacy of Hybrid Expert Routing

RQ4: To what extent does a dynamic “Expert Routing” mechanism enhance overall accuracy compared to static and zero-shot baselines?

Our categorical analysis identified a dual limitation in the standard deployment of Large Language Models: generalist models lack depth in logical tasks. In contrast, code-specialized models suffer from severity overestimation and weak documentation skills. To address these deficiencies simultaneously, we evaluated a Hybrid Expert System.

To rigorously isolate the contribution of our routing strategy, we established two distinct benchmarks based on our findings in RQ1 and RQ3:

1. **Composite Zero-Shot Baseline ($k = 0$):** Calculated as the “instance-weighted average” of the constituent models (Qwen2.5 for Logic/Discussion, Mistral for Documentation) at $k = 0$. This yields a baseline score of 6.21, reflecting the system’s performance without any retrieval context.
2. **Single Best Model (SBM):** Represents the strongest static configuration available. As shown in Table 7, Qwen2.5-Coder-32B ($k = 3$) achieved the highest standalone score (6.53) among individual models. We use this as the reference point to measure the specific value of dynamic routing.

The Hybrid architecture dynamically routes tasks to the specialist best suited for the semantic category:

- **Logic (Functional/Refactoring):** Routed to Qwen2.5-Coder-32B ($k = 3$) to unlock “Latent Knowledge” and ensure AST level correctness.
- **Documentation:** Routed to Mistral-Instruct-7B ($k = 3$) to take advantage of its superior natural language fluency.

Results and Engineering Implications

As presented in Table 9, the Hybrid Expert System achieves a significant performance gain over both baselines.

Crucially, the Hybrid System ($\mu = 7.03$) significantly outperforms the Single Best Model ($\mu = 6.53$). We validated this 7.7% net gain using a paired samples t -test [51]. To avoid the bias of majority classes present in the full test set ($N = 1625$), we performed this analysis on the human verified balanced dataset ($N = 105$, detailed in Section 4.4.3), ensuring equal representation across all semantic categories. The test confirmed that the difference is statistically significant ($t(104) = 4.48, p < 0.001$) with a Cohen’s d of 0.47 [52]. This empirical evidence supports the premise that our hybrid architecture provides superior alignment compared to a monolithic model (SBM).

Table 9. The Hybrid System achieves a Mean Score of 7.03, significantly outperforming the Single Best Model (SBM) by 7.7% and the Zero-Shot baseline by 13.2%.

Category	Zero-Shot Baseline ($k = 0$)	Single Best Model (SBM) ($k = 3$)	Hybrid System (Ours)	Routing Strategy
Functional	6.06	6.76	6.76	Qwen2.5 ($k = 3$)
Refactoring	6.76	7.20	7.20	Qwen2.5 ($k = 3$)
Discussion	7.02	7.36	7.36	Qwen2.5 ($k = 3$)
Documentation	5.00	4.80	6.80	Mistral ($k = 3$)
Mean Score	6.21	6.53	7.03 ***	–
Net Gain	(Reference)	+5.2%	+13.2%	–
Gain vs. SBM	–	(Reference)	+7.7%	–

*** Statistically significant difference ($p < 0.001$) based on paired t -test. **Bold** values indicate the highest scores in each category.

To further isolate the effectiveness of the routing logic from chance, we also evaluated the system against a “Random Routing Baseline”. This baseline randomly routes tasks between the same constituent experts (Qwen2.5-Coder-32B and Mistral-Instruct-7B) with uniform probability ($p = 0.5$). On the full benchmark, the Hybrid System outperforms this Random Baseline ($\mu = 6.14$) by 14.5%. This advantage was similarly verified on the human verified balanced dataset ($N = 105$, detailed in Section 4.4.3), yielding a statistically significant difference ($t(104) = 4.10, p < 0.001$) with a Cohen’s d of 0.40 [52]. This confirms that the routing mechanism provides a robust, systematic advantage over both fixed (SBM) and random policies.

This improvement is driven by two distinct mechanisms:

1. **Contextual Gain (RAG Effect):** In logical tasks (Functional/Refactoring), introducing context ($k = 3$) increased performance compared to the zero-shot baseline (e.g., $6.06 \rightarrow 6.76$). This aligns with our findings in RQ1, confirming that retrieval successfully activates the model’s latent technical knowledge.
2. **Architectural Gain (Routing Effect):** The Single Best Model (Qwen2.5-Coder-32B) exhibits a critical weakness in the “Documentation” category (Score: 5.00), primarily due to the “Error Severity Overestimation” analyzed in Section 5.3.2. By routing these tasks to Mistral-Instruct, our Hybrid System achieves a 42% surge in this category ($4.80 \rightarrow 6.80$). This effectively neutralizes the severity overestimation, ensuring that benign documentation updates are not misclassified as functional defects.

Consequently, the total system improvement of 13.2% (vs. Zero-Shot) is not only a result of adding data, but a product of precise architectural alignment. This suggests that for CI/CD integration, a routed ensemble of smaller, specialized models offers a superior trade-off between computational efficiency and defect detection accuracy compared to a single generalist model.

5.5. RQ5: Misalignment Between N-Gram Metrics and Technical Quality

RQ5: How do traditional n -gram metrics (e.g., BLEU, ROUGE) compare with semantic evaluation methods in assessing the quality of automated code reviews?

To identify a reliable evaluation strategy for automated code review, we examined the relationship between traditional reference-based metrics and semantic evaluation methods. As shown in Table 6, the results reveal an apparent mismatch between the lexical overlap scores and the actual technical usefulness.

5.5.1. Limitations of Lexical Metrics

As shown in Table 6, models such as CodeLlama-13B achieved relatively high ROUGE-L scores (0.111 in $k = 0$) while receiving the lowest Judge Scores (3.54). This gap highlights

a key weakness of reference-based metrics in open-ended tasks. A technically correct code review can be expressed in many valid ways that do not share surface-level n-grams with a reference answer. As a result, BLEU and ROUGE often penalize meaningful semantic variations, making them poor indicators of technical quality in software engineering settings [53].

5.5.2. Semantic Similarity and Technical Correctness

Embedding-based cosine similarity provides a stronger signal than n-gram metrics, but it remains insensitive to logical correctness. For example, “Mistral-Instruct-7B” maintained a high Semantic Similarity score (0.416) even as its Judge Score dropped ($6.01 \rightarrow 5.46$) in a noisy context. This suggests that embeddings capture topical relevance, such as identifying that a comment discusses loops, but fail to distinguish correct solutions from incorrect or hallucinated ones.

5.5.3. Effectiveness of Semantic Evaluation

In contrast, LLM-as-a-Judge scores show a strong correlation with human expert ratings ($r = 0.82, p < 0.001$), as discussed in Section 4.4.3. This indicates that semantic evaluation methods are better suited to complex reasoning tasks, as they focus on the usefulness and correctness of feedback rather than surface-level textual similarity.

In conclusion, the proposed hybrid architecture offers a pragmatic solution for CI/CD integration, effectively solving the “logic-frequency Trade-off” without the need for massive, proprietary models.

5.6. Validation with Human Evaluation

To verify the validity of the automated judge, we assessed the alignment between the LLM-generated scores and the human expert ratings ($n = 105$). While the initial analysis showed a strong linear relationship (Pearson $r = 0.84, p < 0.001$), we expanded the evaluation to verify the stability of the model using rank-based and agreement-specific metrics against the human ground truth.

First, we calculated Spearman’s rank correlation coefficient to evaluate how well the model preserves the ordering established by human experts. The resulting $\rho = 0.87$ confirms that the judge consistently ranks higher-quality responses above lower-quality ones [54].

Second, to measure inter-annotator agreement while accounting for the ordinal nature of the scoring scale (0–10), we utilized “Cohen’s Quadratic Weighted Kappa” [55]. The analysis resulted in $\kappa = 0.78$. Based on the interpretation benchmarks by Landis and Koch [56], this indicates “substantial agreement” between the automated judge and the human evaluators.

These metrics confirm that the performance gains offered by the hybrid system align with the perceptual preferences of the human evaluators (senior software engineers).

5.7. Ablation Study: Standalone Routing Performance

We conducted a standalone ablation study of the categorization module to confirm that the performance gains in comment generation come from our context-aware routing strategy, rather than just the LLM’s baseline capabilities. By isolating this component, we verified that the retrieval mechanism accurately identifies the semantic intent of code changes before the prompt is even constructed.

5.7.1. Experimental Setup

We used the manually labeled dataset of 1273 samples, which we originally curated for the offline SVM training phase described in Section 3.4. We created a stratified split from

this dataset, keeping approximately 20% ($N = 244$) as a test set dedicated to evaluating performance on previously unseen data.

We focused our quantitative analysis on the four primary semantic categories: “Refactoring”, “Functional”, “Discussion”, and “Documentation”. We excluded the “False Positive” category from the aggregate metrics in Table 10 because it represented less than 3% of the test set ($N = 7$). Including such a small sample size would introduce statistical noise; therefore, we discuss these samples qualitatively below instead.

Table 10. Comparison of standalone routing strategies on the test set ($N = 244$). The proposed method demonstrates a 410.2% improvement in Macro-F1 over the baseline.

Strategy	Accuracy	Macro-F1	Weighted-F1	Improvement (Δ)
Fixed Routing (Baseline)	0.344	0.128	0.176	–
Neighbour-Voting (Ours)	0.664	0.653	0.657	+410.2%

Bold values indicate the best-performing strategy.

We compared two routing strategies:

1. **Fixed Routing (Baseline):** We assigned all queries to the most frequently observed category in training (“Refactoring”). This serves as the lower bound for performance.
2. **Neighbour-Voting Routing (Proposed):** We aggregated votes from the top- k ($k = 3$) retrieved historical code changes based on cosine similarity.

5.7.2. Results and Analysis

Table 10 presents the comparative results. Our proposed Neighbour-Voting strategy achieved a Macro-F1 score of 0.653, which represents a 410.2% relative improvement (The relative improvement is calculated as $\Delta = \frac{F1_{\text{Proposed}} - F1_{\text{Baseline}}}{F1_{\text{Baseline}}} \times 100$. Given the baseline’s low performance (0.128), the proportional gain is substantial) over the Fixed Baseline (Macro-F1: 0.128). The baseline predicted the majority class but yielded zero precision and recall for the remaining three categories, confirming that a dynamic routing mechanism is necessary for this task.

To analyze the stability of our classifier, we present the Confusion Matrix in Table 11 and the detailed per-class metrics in Table 12. We observed high specificity across all categories. Importantly, the system successfully distinguishes between “Refactoring” (Precision: 0.73) and “Functional” (Recall: 0.75) changes. This distinction is challenging because both categories often involve code modifications; however, our retrieval mechanism uses the semantic context from file paths and code diffs to resolve this ambiguity effectively.

Table 11. Confusion Matrix of the Neighbour-Voting Routing module (Primary Categories). Rows represent true labels, columns represent predicted labels.

True Category	Predicted Category			
	Discussion	Documentation	Functional	Refactoring
Discussion	34	7	11	9
Documentation	6	31	2	5
Functional	6	1	41	7
Refactoring	8	7	13	56

Bold values indicate correct predictions along the diagonal.

Qualitative Analysis of Outliers: Although we excluded the quantitative analysis for the “False Positive” category due to insufficient sample support ($N = 7$), we analyzed these instances manually. We found that the model misclassified most of them as “Refactoring” (4) or “Functional” (2). These errors highlight a challenge inherent to retrieval-based systems: valid code changes often share a high semantic resemblance with actionable ones,

making them difficult to distinguish in vector space. We expect that incorporating negative sampling in future work will help reduce this overlap.

Table 12. Detailed Classification Report. The system achieves consistent performance across diverse semantic intents.

Category	Precision	Recall	F1-Score	Support
Discussion	0.63	0.56	0.59	61
Documentation	0.67	0.70	0.69	44
Functional	0.61	0.75	0.67	55
Refactoring	0.73	0.67	0.70	84
<i>Macro Avg</i>	0.66	0.67	0.65	244

Italic indicates aggregated metrics; **bold** values highlight overall macro-level performance.

6. Threats to Validity

We identify several potential threats to the validity of our study and discuss the mitigation strategies used to ensure the reliability of our findings.

6.1. Internal Validity

Internal validity concerns factors that might influence the causal relationship between the treatment (RAG/Model Architecture) and the outcome (Review Quality).

- **Data Leakage and Contamination Audit:** A primary concern when evaluating Large Language Models (LLMs) is data contamination—where the test data might have been present in the model’s pre-training corpus. To mitigate this, we selected the “Home Assistant Core” repository, specifically focusing on Pull Requests (PRs) created after the knowledge cutoff dates of the evaluated models. To empirically verify the integrity of the split between the retrieval corpus and the evaluation (test) set, we conducted a content-based contamination audit. We generated SHA-256 signatures for the code-comment pairs in both splits and confirmed zero exact overlaps between the retrieval corpus and the test set. Finally, to facilitate external verification and reproducibility, the exact dataset snapshot used in these experiments has been versioned as “experiment_snapshot_v1.0” in the crc-py-dataset repository (<https://github.com/busraicoz/crc-py-dataset>, accessed on 26 December 2025).
- **Nondeterminism:** LLM generation is inherently stochastic. To reduce run-to-run variance and improve reproducibility, we fixed the decoding configuration in all experiments. This ensures that performance variations are due to architectural differences and retrieval depth (k), rather than random sampling noise.
- **Embedding Limitations:** A potential threat to construct validity is the use of all-MiniLM-L6-v2, which is optimized for natural language. We selected this model because code review comments are primarily natural language explanations. However, we acknowledge that this may miss syntactic nuances in the code diffs. Future iterations of this work could mitigate this by employing code-specific embedding models such as UnixCoder [57] or GraphCodeBERT [58].
- **Retrieval Effectiveness:** Despite the previously mentioned limitation, we validated the practical effectiveness of our retrieval pipeline through the standalone performance of the “Neighbour-Voting Routing” module (Section 5.7). This module achieved a Macro-F1 score of 0.653 using the exact same retrieval inputs, significantly outperforming the zero-shot baseline (+410.2%). This confirms that the retrieved context contains strong semantic signals, supporting our conclusion that the performance degradation at higher depths ($k = 5$) is due to the LLM’s attention bottleneck (Context Collapse) rather than retrieval noise.

6.2. External Validity

External validity relates to the generalizability of our results to other contexts.

- **Language Bias:** Our dataset consists exclusively of Python code from the Home Assistant Core project. While Python is a dominant language in modern development, our findings regarding “Error Severity Overestimation” or “Context Collapse” may not perfectly transfer to statically typed languages (e.g., Java, C++) or low-resource languages. Future work should validate the proposed Hybrid Expert System in a polyglot dataset.
- **Model Selection:** We limited our evaluation to open-weight models (3B–33B parameters) to ensure accessibility and reproducibility. Consequently, our conclusions may not fully apply to proprietary trillion-parameter models like GPT-5 [59] or Claude 4 [60], which may possess different scaling laws regarding context utilization.

7. Conclusions

In this study, we propose a new automated code review system for Python projects. Unlike standard approaches that treat all code changes equally, our system uses a two-step process. First, we classify the code changes into categories, such as “Functional” or “Refactoring”. Then, based on this category, we send the review task to the most suitable language model (expert LLM). We also use Retrieval-Augmented Generation (RAG) to provide relevant historical examples as context.

Our experiments show that adding more context does not constantly improve performance. We found that smaller models often get confused when given too much information, a problem we call “Context Collapse”. We also observed a clear trade-off between logic and fluency. Larger models (like Qwen2.5) are good at finding bugs, but can be too strict on minor style issues. Conversely, smaller models (like Mistral) generate fluent feedback, but sometimes lack technical depth.

To address this gap, we evaluated a hybrid expert system. By separating logical verification from document generation and assigning each task to the appropriate specialist ($k = 3$), we achieved a 13.2% improvement in the success rate over the zero-shot baseline ($k = 0$). This suggests that a modular approach is more suitable for CI/CD integration than relying solely on zero-shot models.

In future work, we plan to address the observed error severity overestimation. The next step is to explore Direct Preference Optimization (DPO) [61] as a way to align model output with developer expectations. Additionally, grounded in our qualitative analysis of “False Positive” outliers, we aim to refine the retrieval stage by implementing negative sampling. This will improve the system’s ability to distinguish between valid code changes and those that are semantically similar, but do not require actionable feedback. Finally, we plan to extend the hybrid framework to agent-based workflows and multilingual settings, which are critical to scaling automated support in diverse software engineering tasks.

Author Contributions: Conceptualization, B.İ. and G.B.; methodology, B.İ.; software, B.İ.; validation, B.İ. and G.B.; formal analysis, B.İ.; investigation, B.İ.; resources, B.İ.; data curation, B.İ.; writing—original draft preparation, B.İ.; writing—review and editing, B.İ. and G.B.; visualization, B.İ.; supervision, G.B.; project administration, B.İ. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The APC was funded by the authors.

Data Availability Statement: The datasets generated and analyzed in this study are publicly available at <https://github.com/busraicoz/crc-py-dataset> (accessed on 26 December 2025).

Conflicts of Interest: Author Büşra İçöz was employed by the company ING. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as potential conflicts of interest.

Appendix A. Code Review System Prompt

To ensure reproducibility, we provide the complete prompt and scoring rubric of the system used for the code review LLMs.

System Prompt Configuration:

Role: You are a senior Python developer tasked with reviewing the following pull request code. Your goal is to provide a technical, constructive, and actionable review comment that is clear and helpful to developers.

Code to Review:

{code_snippet}

Reference Comments (for similar code blocks):

{similar code block 1}

{similar review comment 1}

{similar category 1}

{similar subcategory 1}

{similar code block 2}

{similar review comment 2}

{similar category 2}

{similar subcategory 2}

{similar code block 3}

{similar review comment 3}

{similar category 3}

{similar subcategory 3}

Instructions:

1. Analyze the code snippet carefully.
2. Assign a subcategory to your review, choosing exactly one from:
 - [
 - "functional", "logical", "validation", "resource",
 - "timing", "support issues", "interface",
 - "solution approach", "alternate output", "code organization",
 - "variable naming", "visual representation", "documentation",
 - "design discussion", "question", "praise", "false positive"]

Output Format:

- Review Comment: <your detailed feedback here>
- Subcategory: <one of the 17 subcategories>

Appendix B. LLM-as-a-Judge System Prompt

To ensure reproducibility, the system prompt and scoring rubric used for the LLM-as-a-Judge evaluation (Llama-3-70B) are provided below.

LLM-as-a-Judge System Prompt Configuration:

Role: You are an expert Senior Software Engineer and Code Reviewer. Your task is to evaluate the quality of a generated code review comment compared to a human-written ground truth based on the rubric defined below.

Input Data:

- **Code Diff:** The code changes introduced in the Pull Request.
- **Ground Truth Comment:** The actual comment written by a human expert.
- **Generated Comment:** The review comment generated by the AI model.

Evaluation Criteria (Scoring Rubric 0–10): Please analyze the generated comment and assign a score based on the following scale:

- **10 (Perfect):** Identifies the exact issue, provides a correct fix/suggestion, and matches the technical depth of the ground truth.
- **8–9 (High Quality):** Technically correct and helpful, but may have minor differences in tone or phrasing compared to the ground truth.
- **5–7 (Partial):** Identifies the general topic or intent correctly, but the solution is incomplete, vague, or lacks specific details.
- **1–4 (Failure):** Misses the core issue, provides irrelevant advice, or hallucinates variables/logic not present in the code.
- **0 (Invalid):** Factually incorrect, empty response, or nonsensical output.

Output Format: Provide your response in a valid JSON format containing the score and a brief reasoning:

```
{
  "score": <integer>,
  "reasoning": "<short_explanation>"
}
```

Appendix C. Reproducibility Protocol

Appendix C.1. Computational Resources

All experiments were conducted on a cloud-based GPU instance provided by Run-Pod [62]. The environment was configured with the following specifications to ensure consistent inference latency and context handling:

- **GPU:** Single NVIDIA RTX A6000 GP with 48 GB VRAM.
- **Storage:** 100 GB NVMe(Non-Volatile Memory Express) SSD (to accommodate the vector store and model weights).
- **Runtime:** Linux-based container with CUDA drivers pre-installed.

Appendix C.2. Model Artifacts and Runtime Environment

We utilized the Ollama runtime (v0.12.7) for serving quantized models.

Appendix C.3. Vector Store Configuration

We utilized Qdrant (v1.16.3) as the vector store.

- **Embedding Model:** sentence-transformers/all-MiniLM-L6-v2.
- **Retrieval Scope:** Hybrid search (Project History + External Knowledge).

Appendix D. Data Selection Protocol

“Daily Trending” Repositories

To ensure reproducibility of the repository selection process, we define the “Daily Trending Heuristic” not by the ephemeral GitHub UI “Trending” tab, but by a deterministic query against the GitHub Search API. A repository was selected for the dataset if it satisfied the **Top-k Star Velocity** criterion during the data collection window.

- **Definition:** A Trending repository is defined as one of the top 10 Python repositories by *total star count* that also exhibited active development.
- **API Endpoint:** GET <https://api.github.com/search/repositories>, accessed on 26 December 2025
- **Exact Query Template:**

```
q = language:python
  +is:public
  +archived:false
  +fork:false
  +stars:>1000

sort = stars
order = desc
per_page = 20
```

- **Filtering Thresholds:**
 1. **Language:** Python.
 2. **Activity:** Must have at least one commit pushed within the query window (e.g., pushed:>2024-06-01).
 3. **Popularity Floor:** Minimum 1000 total stars (to filter out low-quality “spam” repositories that often appear in raw trending feeds).
 4. **Exclusions:** Forks and archived repositories were explicitly excluded.

References

1. Bacchelli, A.; Bird, C. Expectations, outcomes, and challenges of modern code review. In *Proceedings of the 2013 International Conference on Software Engineering, San Francisco, CA, USA, 18–26 May 2013*; ICSE '13; IEEE Press: Piscataway, NJ, USA, 2013; pp. 712–721.
2. Sadowski, C.; Söderberg, E.; Church, L.; Sipko, M.; Bacchelli, A. Modern code review: A case study at google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, Gothenburg, Sweden, 27 May–3 June 2018*; ICSE-SEIP '18; pp. 181–190. [CrossRef]
3. Bosu, A.; Carver, J.C. Impact of developer reputation on code review outcomes in OSS projects: An empirical investigation. In *Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, Torino, Italy, 18–19 September 2014*; ESEM '14. [CrossRef]
4. Rahman, F.; Posnett, D.; Devanbu, P. Recalling the “imprecision” of cross-project defect prediction. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, Cary North, CA, USA, 11–16 November 2012*; FSE '12. [CrossRef]
5. Johnson, B.; Song, Y.; Murphy-Hill, E.; Bowdidge, R. Why don't software developers use static analysis tools to find bugs? In *Proceedings of the 2013 35th International Conference on Software Engineering (ICSE), San Francisco, CA, USA, 18–26 May 2013*; pp. 672–681. [CrossRef]
6. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pondé, H.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374. [CrossRef]
7. Zhang, Q.; Fang, C.; Xie, Y.; Zhang, Y.; Yang, Y.; Sun, W.; Yu, S.; Chen, Z. A Survey on Large Language Models for Software Engineering. *arXiv* **2024**, arXiv:2312.15223. [CrossRef]
8. Turzo, A.K.; Faysal, F.; Poddar, O.; Sarker, J.; Iqbal, A.; Bosu, A. Towards Automated Classification of Code Review Feedback to Support Analytics. In *Proceedings of the 2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), New Orleans, LA, USA, 26–27 October 2023*; pp. 1–12. [CrossRef]

9. Fagan, M.E. Design and code inspections to reduce errors in program development. *IBM Syst. J.* **1976**, *15*, 182–211. [\[CrossRef\]](#)
10. Mateos, P.; Bellogín, A. A systematic literature review of recent advances on context-aware recommender systems. *Artif. Intell. Rev.* **2024**, *58*, 20. [\[CrossRef\]](#)
11. Sadman, N.; Ahsan, M.M.; Mahmud, M.A. ADCR: An Adaptive Tool to select “Appropriate Developer for Code Review” based on Code Context. In Proceedings of the 2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), New York, NY, USA, 28–31 October 2020. [\[CrossRef\]](#)
12. Tufano, M.; Watson, C.; Bavota, G.; Penta, M.D.; White, M.; Shyhyvanyk, D. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Trans. Softw. Eng. Methodol.* **2019**, *28*, 1–29. [\[CrossRef\]](#)
13. Li, Z.; Lu, S.; Guo, D.; Duan, N.; Jannu, S.; Jenks, G.; Majumder, D.; Green, J.; Svyatkovskiy, A.; Fu, S.; et al. CodeReviewer: Pre-Training for Automating Code Review Activities. *arXiv* **2022**, arXiv:2203.09095. [\[CrossRef\]](#)
14. Lin, H.Y.; Thongtanunam, P.; Treude, C.; Godfrey, M.W.; Liu, C.; Charoenwet, W. Leveraging Reviewer Experience in Code Review Comment Generation. *ACM Trans. Softw. Eng. Methodol.* **2025**, just accepted. [\[CrossRef\]](#)
15. Rasheed, Z.; Sami, M.A.; Waseem, M.; Kemell, K.K.; Wang, X.; Nguyen, A.; Systä, K.; Abrahamsson, P. AI-powered Code Review with LLMs: Early Results. *arXiv* **2025**, arXiv:2404.18496. [\[CrossRef\]](#)
16. Rybalchenko, A.; Al-Turany, M. Leveraging Large Language Models for Enhanced Code Review. *EPJ Web Conf.* **2025**, *337*, 01066. [\[CrossRef\]](#)
17. Haroon, S.; Khan, A.F.; Humayun, A.; Gill, W.; Amjad, A.H.; Butt, A.R.; Khan, M.T.; Gulzar, M.A. How Accurately Do Large Language Models Understand Code? *arXiv* **2025**, arXiv:2504.04372. [\[CrossRef\]](#)
18. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Proceedings of the 34th International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 6–12 December 2020; NIPS ’20.
19. Hong, H.; Baik, J. Retrieval-Augmented Code Review Comment Generation. *arXiv* **2025**, arXiv:2506.11591. [\[CrossRef\]](#)
20. Wang, Z.Z.; Asai, A.; Yu, X.V.; Xu, F.F.; Xie, Y.; Neubig, G.; Fried, D. CodeRAG-Bench: Can Retrieval Augment Code Generation? In *Proceedings of the Findings of the Association for Computational Linguistics: NAACL 2025*; Chiruzzo, L., Ritter, A., Wang, L., Eds.; Association for Computational Linguistics: Albuquerque, NM, USA, 2025; pp. 3199–3214. [\[CrossRef\]](#)
21. Qdrant: Vector Database for the Next Generation of AI Applications. Available online: <https://qdrant.tech/> (accessed on 23 December 2025).
22. Zhang, Y.; Li, Y.; Cui, L.; Cai, D.; Liu, L.; Fu, T.; Huang, X.; Zhao, E.; Zhang, Y.; Xu, C.; et al. Siren’s Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *arXiv* **2025**, arXiv:2309.01219. [\[CrossRef\]](#)
23. Sentence-Transformers/All-MiniLM-L6-v2: Model Card. Available online: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2> (accessed on 23 December 2025).
24. Manning, C.D.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*; Cambridge University Press: Cambridge, UK, 2008.
25. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 3–7 November 2019; pp. 3982–3992. [\[CrossRef\]](#)
26. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. In Proceedings of the 34th International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 6–12 December 2020; NIPS ’20.
27. Kalliamvakou, E.; Gousios, G.; Blincoe, K.; Singer, L.; German, D.M.; Damian, D. The promises and perils of mining GitHub. In Proceedings of the 11th Working Conference on Mining Software Repositories, Hyderabad, India, 31 May–1 June 2014; MSR 2014; pp. 92–101. [\[CrossRef\]](#)
28. Salton, G.; Buckley, C. Term-weighting approaches in automatic text retrieval. *Inf. Process. Manage.* **1988**, *24*, 513–523. [\[CrossRef\]](#)
29. Cortes, C.; Vapnik, V. Support-Vector Networks. *Mach. Learn.* **1995**, *20*, 273–297. [\[CrossRef\]](#)
30. Triguero, I.; García, S.; Herrera, F. Self-labeled techniques for semi-supervised learning: Taxonomy, software and empirical study. *Knowl. Inf. Syst.* **2015**, *42*, 245–284. [\[CrossRef\]](#)
31. Home Assistant Community. Home-Assistant/Core: Open Source Home Automation That Puts Local Control and Privacy First. Available online: <https://github.com/home-assistant/core> (accessed on 23 December 2025).
32. Home Assistant Developer Docs. Integration Architecture. Available online: https://developers.home-assistant.io/docs/architecture_components/ (accessed on 23 December 2025).
33. Home Assistant Developer Docs. Pull Request Review Process. Available online: <https://developers.home-assistant.io/docs/review-process/> (accessed on 23 December 2025).
34. Open LLM Leaderboard. Hugging Face. Available online: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard (accessed on 22 December 2025).
35. Team, O. Ollama: Large Language Model Runner. 2024. Available online: <https://github.com/ollama/ollama> (accessed on 23 December 2024).

36. Guo, D.; Zhu, Q.; Yang, D.; Xie, Z.; Dong, K.; Zhang, W.; Chen, G.; Bi, X.; Wu, Y.; Li, Y.K.; et al. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv* **2024**, arXiv:2401.14196. [[CrossRef](#)]
37. Hui, B.; Yang, J.; Cui, Z.; Yang, J.; Liu, D.; Zhang, L.; Liu, T.; Zhang, J.; Yu, B.; Lu, K.; et al. Qwen2.5-Coder Technical Report. *arXiv* **2024**, arXiv:2409.12186. [[CrossRef](#)]
38. Mistral AI Team. Codestral: Hello, World! 2024. Available online: <https://mistral.ai/news/codestral/> (accessed on 23 December 2025).
39. Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.; Adi, Y.; Liu, J.; Remez, T.; Rapin, J.; et al. Code Llama: Open Foundation Models for Code. *arXiv* **2023**, arXiv:2308.12950.
40. Jiang, A.Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D.S.; de las Casas, D.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; et al. Mistral 7B. *arXiv* **2023**, arXiv:2310.06825. . [[CrossRef](#)]
41. Abdin, M.; Aneja, J.; Awadalla, H.; Awadallah, A.; Awan, A.A.; Bach, N.; Bahree, A.; Bakhtiari, A.; Bao, J.; Behl, H.; et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. *arXiv* **2024**, arXiv:2404.14219. . [[CrossRef](#)]
42. Papineni, K.; Roukos, S.; Ward, T.; Zhu, W.J. Bleu: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*; Isabelle, P., Charniak, E., Lin, D., Eds.; Association for Computational Linguistics: Philadelphia, PA, USA, 2002; pp. 311–318. [[CrossRef](#)]
43. Lin, C.Y. ROUGE: A Package for Automatic Evaluation of Summaries. In *Proceedings of the Text Summarization Branches Out*, Barcelona, Spain, 25–26 July 2004; pp. 74–81.
44. Post, M. A Call for Clarity in Reporting BLEU Scores. In *Proceedings of the Conference on Machine Translation*, Brussels, Belgium, 31 October–1 November 2018.
45. Zheng, L.; Chiang, W.L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.P.; et al. Judging LLM-as-a-judge with MT-bench and Chatbot Arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 10–16 December 2023; NIPS '23.
46. Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. The Llama 3 Herd of Models. *arXiv* **2024**, arXiv:2407.21783. [[CrossRef](#)]
47. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 6–12 December 2022; NIPS '22.
48. Hu, M.; Wu, H.; Guan, Z.; Zhu, R.; Guo, D.; Qi, D.; Li, S. No Free Lunch: Retrieval-Augmented Generation Undermines Fairness in LLMs, Even for Vigilant Users. *arXiv* **2024**, arXiv:2410.07589. [[CrossRef](#)]
49. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; Liang, P. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 157–173. [[CrossRef](#)]
50. Askell, A.; Bai, Y.; Chen, A.; Drain, D.; Ganguli, D.; Henighan, T.; Jones, A.; Joseph, N.; Mann, B.; DasSarma, N.; et al. A General Language Assistant as a Laboratory for Alignment. *arXiv* **2021**, arXiv:2112.00861. [[CrossRef](#)]
51. Dietterich, T.G. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. *Neural Comput.* **1998**, *10*, 1895–1923. [[CrossRef](#)]
52. Cohen, J. *Statistical Power Analysis for the Behavioral Sciences*; Lawrence Erlbaum Associates: Mahwah, NJ, USA, 1988.
53. Ren, S.; Guo, D.; Lu, S.; Zhou, L.; Liu, S.; Tang, D.; Sundaresan, N.; Zhou, M.; Blanco, A.; Ma, S. CodeBLEU: A Method for Automatic Evaluation of Code Synthesis. *arXiv* **2020**, arXiv:2009.10297. [[CrossRef](#)]
54. Spearman, C. The proof and measurement of association between two things. *Int. J. Epidemiol.* **2010**, *39*, 1137–1150. [[CrossRef](#)]
55. Cohen, J. Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychol. Bull.* **1968**, *70*, 213–220. [[CrossRef](#)]
56. Landis, J.R.; Koch, G.G. The measurement of observer agreement for categorical data. *Biometrics* **1977**, *33*, 159–174. [[CrossRef](#)]
57. Guo, D.; Lu, S.; Duan, N.; Wang, Y.; Zhou, M.; Yin, J. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*; Muresan, S., Nakov, P., Villavicencio, A., Eds.; Association for Computational Linguistics: Dublin, Ireland, 2022; pp. 7212–7225. [[CrossRef](#)]
58. Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S.; et al. GraphCodeBERT: Pre-training Code Representations with Data Flow. *arXiv* **2020**, arXiv:2009.08366. [[CrossRef](#)]
59. OpenAI. Introducing GPT-5. OpenAI Blog, 2025. Available online: <https://openai.com/index/introducing-gpt-5> (accessed on 26 December 2025).
60. Anthropic. Introducing Claude 4. Anthropic News. 2025. Available online: <https://www.anthropic.com/news/claude-4> (accessed on 26 December 2025).

61. Rafailov, R.; Sharma, A.; Mitchell, E.; Ermon, S.; Manning, C.D.; Finn, C. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. *arXiv* **2024**, arXiv:2305.18290. [[CrossRef](#)]
62. RunPod: GPU Cloud for AI. Available online: <https://console.runpod.io> (accessed on 15 January 2026).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.